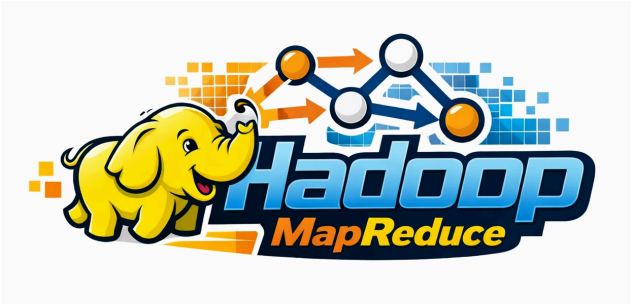
TP 2



Dans Hadoop, les clés sont des objets qui implémentent l'interface WritableComparable et les valeurs sont des objets qui implémentent l'interface Writable. Hadoop propose les types de données suivants :

- IntWritable (int Java)
- LongWritable (long Java)
- FloatWritable (float Java)
- DoubleWritable (double Java)
- Text (String Java)

Exercice 1) Programmation du MapReduce pour l'exemple du WordCount

Nous allons programmer l'algorithme de MapReduce à travers un exemple très connu : le WordCount.

1. Récupérez le fichier contenant l'intégralité de l'oeuvre de Shakespeare et sauvegardez-le sous HDFS dans le répertoire "gutenberg".
2. Avec l'IDE de votre choix, créez un projet Java nommé "MapReduce" en y ajoutant les librairies suivantes : "hadoop-common-3.2.1.jar" et "hadoop-mapreduce-client-core-3.2.1.jar".
3. Dans le projet "MapReduce", créez un package "wc".
4. Créez successivement le driver, le mapper et le reducer dans le package "wc" (code en fin de ce fichier).

Attention, les sources ".java" doivent être compilés en Java 8 !

5. Générez une archive java ".jar" qui doit contenir les classes précédemment créées : *jar cvf wc.jar wc* ("wc" est le répertoire qui contient les fichiers ".class").

6. Copiez le ficher "jar" sur le namenode.
7. Lancez le job Hadoop : *hadoop jar wc.jar wc.WcDriver gutenberg wordcounts*
8. Vérifiez le résultat contenu dans le fichier "part-r-00000" situé dans le répertoire "wordcounts".

Exercice 2) Réaliser un programme MapReduce qui recherche les listes d'anagramme dans un texte.

1. Récupérez le fichier "liste_mots.txt" et sauvegardez-le sous Hadoop dans le répertoire "mots".
2. Réalisez un programme MapReduce qui permet de trouver tous les anagrammes contenus dans cette liste de mots.

Exercice 3) **Réaliser un programme qui calcule le nombre de hits par quart d'heure entre 10h30 et 23h15 à partir des logs d'un site.**

1. Récupérez le fichier "cloudera_log.txt" (38 Mo) et sauvegardez-le sous HDFS dans le répertoire "logs".
2. Réalisez un programme MapReduce qui calcule le nombre de hits (requêtes HTTP) par quart d'heure entre 10h30 et 23h15.

Exercice 4) **Réaliser un programme MapReduce qui indexe un mot dans un texte.**

1. Récupérez le fichier "The_adventures_of_Sherlock_Holmes".
2. Réalisez un programme MapReduce qui permet de créer l'index d'un mot passé en paramètre de votre commande par rapport au texte ci-dessus.

Par exemples, pour les mots "burglar", "murderer", "bachelor", "gentleman" et "innocent" l'index créé possède respectivement 1, 5, 8, 45, 17 entrée(s).

Conseils pratiques

Pour **insérer le numéro des lignes dans un fichier texte**, utilisez la commande shell ci-dessous.

cat -n "fichier_source" > "fichier_resultat"

Dans votre classe Driver, pensez que votre **commande** contiendra **trois arguments** : le répertoire source, le répertoire de sortie et le mot à indexer. Par exemple :

hadoop jar index.jar index.IndexDriver <rep_in> <rep_out> <mot_a_indexer>

Pour passer le mot à indexer de la classe Driver vers la classe Mapper, ajoutez les lignes suivantes :

Dans le Driver

```
// Ajout de mot à indexer au contexte des jobs.
job.getConfiguration().set("mot_a_indexer", args[2]);
```

Dans le Mapper

```
// Lecture du mot à indexer.
String mot = context.getConfiguration().get("mot_a_indexer");
```

Exercice 5) Soit un fichier de logs donc chaque enregistrement a le format ci-dessous :

```
id_cde;id_client;id_prod_1;id prod_2; … ;id_prod_n
```

3. Pour tout couple de produit (x, y), combien de fois a-t-il été commandé par des clients différents ?

Sites sur les expressions régulières

- Learn Regular Expressions with simple, interactive exercises
- Les expressions régulières avec l'API Regex de Java
- Regexr pour tester vos expressions régulières. Vous pouvez aussi utiliser NotePad++

WcDriver.java

```
package wc;

import org.apache.hadoop.fs.Path;
import org.apache.hadoop.io.IntWritable;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.Job;
import org.apache.hadoop.mapreduce.lib.input.FileInputFormat;
import org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;

public class WcDriver {

public static void main(String[] args) throws Exception {

    // Vérification de la présence des 2 arguments de la fonction.
    if (args.length != 2) {
        System.out.printf("Format de la ligne de commande WcDriver <rep source> <rep cible>\n");
        System.exit(-1);
    }

    // Instanciation et initialisation du job Hadoop.
    Job job = Job.getInstance();
    job.setJarByClass(WcDriver.class); // Classe de lancement.
    job.setJobName("Word Count"); // Nom du job.
    FileInputFormat.setInputPaths(job, new Path(args[0])); // Chemin du répertoire d'entrée.
    FileOutputFormat.setOutputPath(job, new Path(args[1])); // Chemin du répertoire de sortie.

    // Indication au job des classes à exécuter pour le mapper et le reducer.
    job.setMapperClass(WcMapper.class);
    job.setReducerClass(WcReducer.class);

    // Indication des types pour le couple (key, value) en sortie du mapper.
    job.setMapOutputKeyClass(Text.class);
    job.setMapOutputValueClass(IntWritable.class);

    // Indication des types pour le couple (key, value) en sortie du reducer.
    job.setOutputKeyClass(Text.class);
    job.setOutputValueClass(IntWritable.class);

    // Lancement du job.
    boolean ok = job.waitForCompletion(true);
    System.exit(ok ? 0 : 1);
}
}
```

WcMapper.java

```
package wc;

import java.io.IOException;
import org.apache.hadoop.io.IntWritable;
import org.apache.hadoop.io.LongWritable;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.Mapper;

public class WcMapper extends Mapper<LongWritable, Text, // Ne pas modifier !
                                      Text, IntWritable> // Couple (key, value) en sortie du mapper
{
// key est l'offset, adresse relative depuis le début du fichier (Ne pas modifier !).
// value est le texte à mapper (Ne pas modifier !).
public void map (LongWritable key, Text value, Context context)
                                             throws IOException, InterruptedException {

    // Transforme l'objet 'Text' en chaîne de caractères.
    String line = value.toString();

    // Découpe la chaîne de caractères.
    for (String word : line.split("\\W+"))
        if (word.length() > 0)
            // Ecriture du couple clé/valeur en sortie du mapper.
            context.write(new Text(word), new IntWritable(1));
}
}
```

WcReducer.java

```
package wc;

import java.io.IOException;
import org.apache.hadoop.io.IntWritable;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.Reducer;

public class WcReducer extends Reducer<Text, IntWritable, // Couple (key, value) en sortie du
mapper
                                       Text, IntWritable> // Couple (key, value) en sortie du
                                       reducer
{
// key est la clé extraite du mapper.
// values est le liste des valeurs issues des mappers pour chaque occurrence de key.
public void reduce (Text key, Iterable<IntWritable> values, Context context)
                                             throws IOException, InterruptedException {

    int wordcount = 0;

    // On compte le nombre de valeurs.
    for (IntWritable value : values)
       wordcount = wordcount + value.get();

    // Ecriture du résultat
    context.write(key, new IntWritable(wordcount));
}
}
```